BY BOBBY ILIEV

Introduction to Bash Scripting

FOR DEVELOPERS

Table of Contents

About the book **8**
About the author 9
Sponsors 10
Ebook PDF Generation Tool 12
Ebook ePub Generation Tool 13
Book Cover 14
License 15

Introduction to Bash scripting **16**

Bash Structure **17**

Bash Hello World **19**

Bash Variables **21**

Bash User Input **25**

Bash Comments **27**

Bash Arguments **28**

Bash Arrays **31**
Array Slicing 33
String Slicing 34

Bash Conditional Expressions **36**
File expressions 37
String expressions 39
Arithmetic operators 41
Exit status operators 43

Bash Conditionals **44**
If statement 45
If Else statement 46
Switch case statements 49
Conclusion 51

Bash Loops **52**
For loops 53
While loops 55
Until Loops 57
Continue and Break 58

Bash Functions **61**

Debugging, testing and shortcuts **63**

Creating custom bash commands **66**
Example 67
Making the change persistent 69
Listing all of the available aliases 70
Conclusion 71

Write your first Bash script **72**

Planning the script 73
Writing the script 74
Adding comments 75
Adding your first variable 76
Adding your first function 77
Adding more functions challenge 79
The sample script 80
Conclusion 82

Creating an interactive menu in Bash 83
Planning the functionality 84
Adding some colors 86

Adding the menu 87
Testing the script 89
Conclusion 92

Executing BASH scripts on Multiple Remote Servers 93
Prerequisites 94
The BASH Script 95
Running the Script on all Servers 97
Conclusion 98

Work with JSON in BASH using jq 99
Planning the script 100
Installing jq 101
Parsing JSON with jq 103
Getting the first element with jq 105
Getting a value only for specific key 106
Using jq in a BASH script 107

Conclusion 110

Working with Cloudflare API with Bash 111

Prerequisites 112

Challenge - Script requirements 113

Example script 114

Conclusion 116

BASH Script parser to Summarize Your NGINX and Apache Access Logs 117

Script requirements 118

Example script 119

Running the script 120

Understanding the output 121

Conclusion 122

Sending emails with Bash and SSMTP 123

Prerequisites 124

Installing SSMTP 125

Configuring SSMTP 126

Sending emails with SSMTP 127

Sending A File with SSMTP (optional) 128

Conclusion 129

Password Generator Bash Script 130

:warning: Security 131

Script summary 132

Prerequisites 133

Generate a random password 134

The script 136
The full script: 137
Conclusion 138
Contributed by 139

Redirection in Bash 140

Difference between Pipes and Redirections 141

Redirection in Bash 142
STDIN (Standard Input) 143
STDOUT (Standard Output) 145
STDERR (Standard Error) 147

Piping 149

HereDocument 151

HereString 153
Summary 154

Automatic WordPress on LAMP installation with BASH 155

Prerequisites 156

Planning the functionality 157

The script 159

The full script 166

Summary 170

Wrap Up **171**

About the book

- **This version was published on 02 08 2024**

This is an open-source introduction to Bash scripting guide that will help you learn the basics of Bash scripting and start writing awesome Bash scripts that will help you automate your daily SysOps, DevOps, and Dev tasks. No matter if you are a DevOps/SysOps engineer, developer, or just a Linux enthusiast, you can use Bash scripts to combine different Linux commands and automate tedious and repetitive daily tasks so that you can focus on more productive and fun things.

The guide is suitable for anyone working as a developer, system administrator, or a DevOps engineer and wants to learn the basics of Bash scripting.

The first 13 chapters would be purely focused on getting some solid Bash scripting foundations, then the rest of the chapters would give you some real-life examples and scripts.

About the author

My name is Bobby Iliev, and I have been working as a Linux DevOps Engineer since 2014. I am an avid Linux lover and supporter of the open-source movement philosophy. I am always doing that which I cannot do in order that I may learn how to do it, and I believe in sharing knowledge.

I think it's essential always to keep professional and surround yourself with good people, work hard, and be nice to everyone. You have to perform at a consistently higher level than others. That's the mark of a true professional.

For more information, please visit my blog at https://bobbyiliev.com, follow me on Twitter @bobbyiliev_ and YouTube.

Sponsors

This book is made possible thanks to these fantastic companies!

Materialize

The Streaming Database for Real-time Analytics.

Materialize is a reactive database that delivers incremental view updates. Materialize helps developers easily build with streaming data using standard SQL.

DigitalOcean

DigitalOcean is a cloud services platform delivering the simplicity developers love and businesses trust to run production applications at scale.

It provides highly available, secure, and scalable compute, storage, and networking solutions that help developers build great software faster.

Founded in 2012 with offices in New York and Cambridge, MA, DigitalOcean offers transparent and affordable pricing, an elegant user interface, and one of the largest libraries of open source resources available.

For more information, please visit https://www.digitalocean.com or follow @digitalocean on Twitter.

If you are new to DigitalOcean, you can get a free $200 credit and spin up your own servers via this referral link here:

Free $200 Credit For DigitalOcean

DevDojo

The DevDojo is a resource to learn all things web development and web design. Learn on your lunch break or wake up and enjoy a cup of coffee with us to learn something new.

Join this developer community, and we can all learn together, build together, and grow together.

Join DevDojo

For more information, please visit https://www.devdojo.com or follow @thedevdojo on Twitter.

Ebook PDF Generation Tool

This ebook was generated by Ibis developed by Mohamed Said.

Ibis is a PHP tool that helps you write eBooks in markdown.

Ebook ePub Generation Tool

The ePub version was generated by Pandoc.

Book Cover

The cover for this ebook was created with Canva.com.

If you ever need to create a graphic, poster, invitation, logo, presentation – or anything that looks good — give Canva a go.

License

MIT License

Copyright (c) 2020 Bobby Iliev

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Introduction to Bash scripting

Welcome to this Bash basics training guide! In this **bash crash course**, you will learn the **Bash basics** so you could start writing your own Bash scripts and automate your daily tasks.

Bash is a Unix shell and command language. It is widely available on various operating systems, and it is also the default command interpreter on most Linux systems.

Bash stands for Bourne-Again SHell. As with other shells, you can use Bash interactively directly in your terminal, and also, you can use Bash like any other programming language to write scripts. This book will help you learn the basics of Bash scripting including Bash Variables, User Input, Comments, Arguments, Arrays, Conditional Expressions, Conditionals, Loops, Functions, Debugging, and testing.

Bash scripts are great for automating repetitive workloads and can help you save time considerably. For example, imagine working with a group of five developers on a project that requires a tedious environment setup. In order for the program to work correctly, each developer has to manually set up the environment. That's the same and very long task (setting up the environment) repeated five times at least. This is where you and Bash scripts come to the rescue! So instead, you create a simple text file containing all the necessary instructions and share it with your teammates. And now, all they have to do is execute the Bash script and everything will be created for them.

In order to write Bash scripts, you just need a UNIX terminal and a text editor like Sublime Text, VS Code, or a terminal-based editor like vim or nano.

Bash Structure

Let's start by creating a new file with a `.sh` extension. As an example, we could create a file called `devdojo.sh`.

To create that file, you can use the `touch` command:

```
touch devdojo.sh
```

Or you can use your text editor instead:

```
nano devdojo.sh
```

In order to execute/run a bash script file with the bash shell interpreter, the first line of a script file must indicate the absolute path to the bash executable:

```
#!/bin/bash
```

This is also called a Shebang.

All that the shebang does is to instruct the operating system to run the script with the `/bin/bash` executable.

However, bash is not always in `/bin/bash` directory, particularly on non-Linux systems or due to installation as an optional package. Thus, you may want to use:

```
#!/usr/bin/env bash
```

It searches for bash executable in directories, listed in PATH environmental variable.

Bash Hello World

Once we have our `devdojo.sh` file created and we've specified the bash shebang on the very first line, we are ready to create our first `Hello World` bash script.

To do that, open the `devdojo.sh` file again and add the following after the `#!/bin/bash` line:

```
#!/bin/bash

echo "Hello World!"
```

Save the file and exit.

After that make the script executable by running:

```
chmod +x devdojo.sh
```

After that execute the file:

```
./devdojo.sh
```

You will see a "Hello World" message on the screen.

Another way to run the script would be:

```
bash devdojo.sh
```

As bash can be used interactively, you could run the following command directly in your terminal and you would get the same result:

```
echo "Hello DevDojo!"
```

Putting a script together is useful once you have to combine multiple commands together.

Bash Variables

As in any other programming language, you can use variables in Bash Scripting as well. However, there are no data types, and a variable in Bash can contain numbers as well as characters.

To assign a value to a variable, all you need to do is use the `=` sign:

```
name="DevDojo"
```

> **Notice:** as an important note, you can not have spaces before and after the `=` sign.

After that, to access the variable, you have to use the `$` and reference it as shown below:

```
echo $name
```

Wrapping the variable name between curly brackets is not required, but is considered a good practice, and I would advise you to use them whenever you can:

```
echo ${name}
```

The above code would output: `DevDojo` as this is the value of our `name` variable.

Next, let's update our `devdojo.sh` script and include a variable in it.

Again, you can open the file `devdojo.sh` with your favorite text editor, I'm using nano here to open the file:

```
nano devdojo.sh
```

Adding our `name` variable here in the file, with a welcome message. Our file now looks like this:

```
#!/bin/bash

name="DevDojo"

echo "Hi there $name"
```

Save it and run the file using the command below:

```
./devdojo.sh
```

You would see the following output on your screen:

```
Hi there DevDojo
```

Here is a rundown of the script written in the file:

- `#!/bin/bash` - At first, we specified our shebang.
- `name=DevDojo` - Then, we defined a variable called `name` and assigned a value to it.
- `echo "Hi there $name"` - Finally, we output the content of the variable on the screen as a welcome message by using `echo`

You can also add multiple variables in the file as shown below:

```
#!/bin/bash

name="DevDojo"
greeting="Hello"

echo "$greeting $name"
```

Save the file and run it again:

```
./devdojo.sh
```

You would see the following output on your screen:

```
Hello DevDojo
```

Note that you don't necessarily need to add semicolon `;` at the end of each line. It works both ways, a bit like other programming language such as JavaScript!

You can also add variables in the Command Line outside the Bash script and they can be read as parameters:

```
./devdojo.sh Bobby buddy!
```

This script takes in two parameters `Bobby`and `buddy!` separated by space. In the `devdojo.sh` file we have the following:

```
#!/bin/bash

echo "Hello there" $1
```

`$1` is the first input (`Bobby`) in the Command Line. Similarly, there could be more inputs and they are all referenced to by the `$` sign and their respective order of input. This means that `buddy!` is referenced to using `$2`. Another useful method for reading variables is the `$@` which reads all inputs.

So now let's change the `devdojo.sh` file to better understand:

```
#!/bin/bash

echo "Hello there" $1

# $1 : first parameter

echo "Hello there" $2

# $2 : second parameter

echo "Hello there" $@

# $@ : all
```

The output for:

```
./devdojo.sh Bobby buddy!
```

Would be the following:

```
Hello there Bobby
Hello there buddy!
Hello there Bobby buddy!
```

Bash User Input

With the previous script, we defined a variable, and we output the value of the variable on the screen with the `echo $name`.

Now let's go ahead and ask the user for input instead. To do that again, open the file with your favorite text editor and update the script as follows:

```
#!/bin/bash

echo "What is your name?"
read name

echo "Hi there $name"
echo "Welcome to DevDojo!"
```

The above will prompt the user for input and then store that input as a string/text in a variable.

We can then use the variable and print a message back to them.

The output of the above script would be:

- First run the script:

```
./devdojo.sh
```

- Then, you would be prompted to enter your name:

```
What is your name?
Bobby
```

- Once you've typed your name, just hit enter, and you will get the following output:

```
Hi there Bobby
Welcome to DevDojo!
```

To reduce the code, we could change the first `echo` statement with the `read -p`, the `read` command used with `-p` flag will print a message before prompting the user for their input:

```
#!/bin/bash

read -p "What is your name? " name

echo "Hi there $name"
echo "Welcome to DevDojo!"
```

Make sure to test this out yourself as well!

Bash Comments

As with any other programming language, you can add comments to your script. Comments are used to leave yourself notes through your code.

To do that in Bash, you need to add the # symbol at the beginning of the line. Comments will never be rendered on the screen.

Here is an example of a comment:

```
# This is a comment and will not be rendered on the screen
```

Let's go ahead and add some comments to our script:

```
#!/bin/bash

# Ask the user for their name

read -p "What is your name? " name

# Greet the user
echo "Hi there $name"
echo "Welcome to DevDojo!"
```

Comments are a great way to describe some of the more complex functionality directly in your scripts so that other people could find their way around your code with ease.

Bash Arguments

You can pass arguments to your shell script when you execute it. To pass an argument, you just need to write it right after the name of your script. For example:

```
./devdojo.com your_argument
```

In the script, we can then use `$1` in order to reference the first argument that we specified.

If we pass a second argument, it would be available as `$2` and so on.

Let's create a short script called `arguments.sh` as an example:

```
#!/bin/bash

echo "Argument one is $1"
echo "Argument two is $2"
echo "Argument three is $3"
```

Save the file and make it executable:

```
chmod +x arguments.sh
```

Then run the file and pass **3** arguments:

```
./arguments.sh dog cat bird
```

The output that you would get would be:

```
Argument one is dog
Argument two is cat
Argument three is bird
```

To reference all arguments, you can use `$@`:

```
#!/bin/bash

echo "All arguments: $@"
```

If you run the script again:

```
./arguments.sh dog cat bird
```

You will get the following output:

```
All arguments: dog cat bird
```

Another thing that you need to keep in mind is that `$0` is used to reference the script itself.

This is an excellent way to create self destruct the file if you need to or just get the name of the script.

For example, let's create a script that prints out the name of the file and deletes the file after that:

```
#!/bin/bash

echo "The name of the file is: $0 and it is going to be self-deleted."

rm -f $0
```

You need to be careful with the self deletion and ensure that you have your script backed up before you self-delete it.

Bash Arrays

If you have ever done any programming, you are probably already familiar with arrays.

But just in case you are not a developer, the main thing that you need to know is that unlike variables, arrays can hold several values under one name.

You can initialize an array by assigning values divided by space and enclosed in `()`. Example:

```
my_array=("value 1" "value 2" "value 3" "value 4")
```

To access the elements in the array, you need to reference them by their numeric index.

> **Notice:** keep in mind that you need to use curly brackets.

- Access a single element, this would output: `value 2`

```
echo ${my_array[1]}
```

- This would return the last element: `value 4`

```
echo ${my_array[-1]}
```

- As with command line arguments using `@` will return all elements in the array, as follows: `value 1 value 2 value 3 value 4`

```
echo ${my_array[@]}
```

- Prepending the array with a hash sign (`#`) would output the total number of elements in the array, in our case it is `4`:

```
echo ${#my_array[@]}
```

Make sure to test this and practice it at your end with different values.

Array Slicing

While Bash doesn't support true array slicing, you can achieve similar results using a combination of array indexing and string slicing:

```
#!/bin/bash

array=("A" "B" "C" "D" "E")

# Print entire array
echo "${array[@]}"  # Output: A B C D E

# Access a single element
echo "${array[1]}"  # Output: B

# Print a range of elements (requires Bash 4.0+)
echo "${array[@]:1:3}"  # Output: B C D

# Print from an index to the end
echo "${array[@]:3}"  # Output: D E
```

When working with arrays, always use `[@]` to refer to all elements, and enclose the parameter expansion in quotes to preserve spaces in array elements.

String Slicing

In Bash, you can extract portions of a string using slicing. The basic syntax is:

```
${string:start:length}
```

Where:

- `start` is the starting index (0-based)
- `length` is the maximum number of characters to extract

Let's look at some examples:

```
#!/bin/bash

text="ABCDE"

# Extract from index 0, maximum 2 characters
echo "${text:0:2}"  # Output: AB

# Extract from index 3 to the end
echo "${text:3}"    # Output: DE

# Extract 3 characters starting from index 1
echo "${text:1:3}"  # Output: BCD

# If length exceeds remaining characters, it stops at the end
echo "${text:3:3}"  # Output: DE (only 2 characters available)
```

Note that the second number in the slice notation represents the maximum length of the extracted substring, not the ending index. This is different from some other programming languages like Python. In Bash, if you specify a length that would extend beyond the end of the string, it will simply stop at the end of the string without raising an error.

For example:

```
text="Hello, World!"

# Extract 5 characters starting from index 7
echo "${text:7:5}"  # Output: World

# Attempt to extract 10 characters starting from index 7
# (even though only 6 characters remain)
echo "${text:7:10}"  # Output: World!
```

In the second example, even though we asked for 10 characters, Bash only returns the 6 available characters from index 7 to the end of the string. This behavior can be particularly useful when you're not sure of the exact length of the string you're working with.

This is a sample from "Introduction to Bash Scripting" by Bobby Iliev.

For more information, Click here.